StickForStats: Master Implementation Plan

Roadmap to the World's Best Statistics Education & Analysis Platform

Document Version: 1.0 **Created:** December 12, 2025 **Last Updated:** December 12, 2025 **Status:** Active Development

---

Table of Contents

---

1. Executive Summary

Mission Statement

Transform StickForStats from an excellent statistics education platform into the world's definitive resource for learning, performing, and publishing statistical analyses.

Key Differentiators to Achieve

1. **AI-First Approach**: Natural language statistical guidance
2. **Publication-Ready Output**: One-click methods sections and reports
3. **Comprehensive Meta-Analysis**: Fill the gap in free meta-analysis tools
4. **Integrated Learning**: Seamless education + analysis workflow
5. **Global Accessibility**: Multi-language, mobile, offline support

Current Competitive Position

Platform	Strengths	Our Advantage
G*Power	Power analysis standard	Web-based + AI guidance + education
JASP	Beautiful Bayesian	Guardian system + 50-decimal precision
SPSS	Industry standard	Free + modern + educational
jamovi	R integration	Better UI + integrated learning
GraphPad	Biomedical focus	Broader scope + AI advisor

---

2. Current State Assessment

2.1 Existing Modules (Production-Ready)

Educational Modules (38+ Interactive Lessons)

Module	Lessons	Lines of Code	Status
Power Analysis	11	~12,050	Complete
PCA (Principal Components)	10	~8,500	Complete
Confidence Intervals	8	~6,200	Complete
Design of Experiments	8	~7,100	Complete
Probability Distributions	6	~4,800	Complete
Statistical Quality Control	6	~5,200	Complete
TOTAL	49	~43,850	Complete

Analysis Capabilities

- **46 Statistical Tests** with Guardian validation
- **50-Decimal Precision** calculations (industry-leading)
- **5 Visualization Types**: Distribution, Relationship, Comparison, Time Series, Composition
- **Advanced Statistics**: ANOVA variants, MANOVA, repeated measures, post-hoc tests
- **Machine Learning**: Regression, classification, clustering

Technical Infrastructure

```
Frontend:
  - React 18 + Material-UI 5
  - D3.js, Three.js (3D), Recharts, Plotly
  - WebSocket support
  - MathJax for formulas

Backend:
  - Django 4.x + REST Framework
  - mpmath (high-precision math)
  - NumPy, SciPy, Statsmodels, Scikit-learn
  - WebSocket channels
```

2.2 Key File Locations

```
/frontend/src/
├── components/
│   ├── statistical-analysis/     # Main analysis hub (13,722 lines)
│   │   ├── StatisticalAnalysisHub.jsx
│   │   ├── modules/              # 7 analysis modules
│   │   └── utils/                # Utility functions
│   ├── power-analysis/           # Power analysis module
│   │   └── education/            # 11 lessons + simulations
│   ├── pca/                      # PCA module
│   │   └── education/            # 10 lessons
│   ├── confidence_intervals/     # CI module
│   │   └── education/            # 8 lessons
│   ├── doe/                      # Design of Experiments
│   │   └── education/            # 8 lessons
│   ├── probability_distributions/# Probability module
│   │   └── education/            # 6 lessons
│   ├── sqc/                      # Statistical Quality Control
│   │   └── education/            # 6 lessons + case studies
│   ├── Guardian/                 # Assumption validation system
│   └── common/                   # Shared components
├── pages/                        # 30+ page components
├── context/                      # Global state
└── App.jsx                       # Main routing (70+ routes)

/backend/
├── core/                         # Core services
│   ├── power_analysis.py
│   ├── assumption_service.py
│   ├── interpretation_service.py
│   └── [54+ service files]
├── stickforstats/                # Django settings
└── [module directories]          # Module-specific backends
```

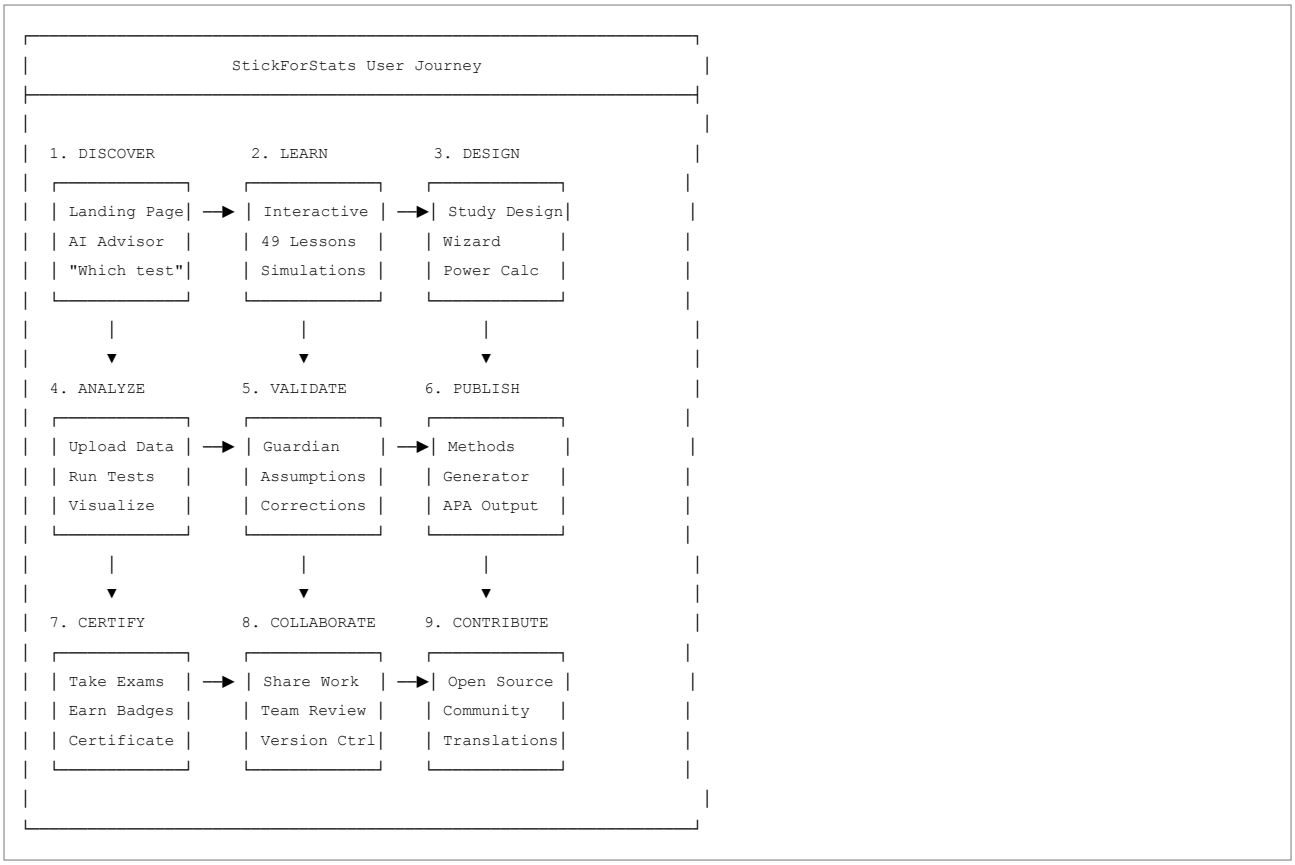
2.3 Gaps to Address

Gap	Current State	Target State
AI Guidance	None	Natural language advisor
Publication Output	Basic export	APA/AMA methods generator
Meta-Analysis	None	Full meta-analysis suite
Study Design	Power analysis only	Complete design wizard
Certification	None	Bronze/Silver/Gold tiers
Mobile	Responsive only	Native mobile app
Languages	English only	6+ languages
Code Export	None	R/Python generation

---

3. Strategic Vision

3.1 The Ultimate User Journey



3.2 Target User Segments

Segment	Needs	Priority Features
Graduate Students	Learn + Analyze + Publish	Education, Methods Gen, Certification
Researchers	Analyze + Publish + Validate	AI Advisor, Meta-Analysis, Guardian
Industry Analysts	Quick Analysis + Reports	AI Advisor, Code Export, Dashboards
Professors	Teach + Grade + Track	LMS Integration, Certification
Regulatory Scientists	Compliance + Precision	50-decimal, Audit Trail, Validation

4. Feature Implementation Plans

4.1 AI Statistical Advisor

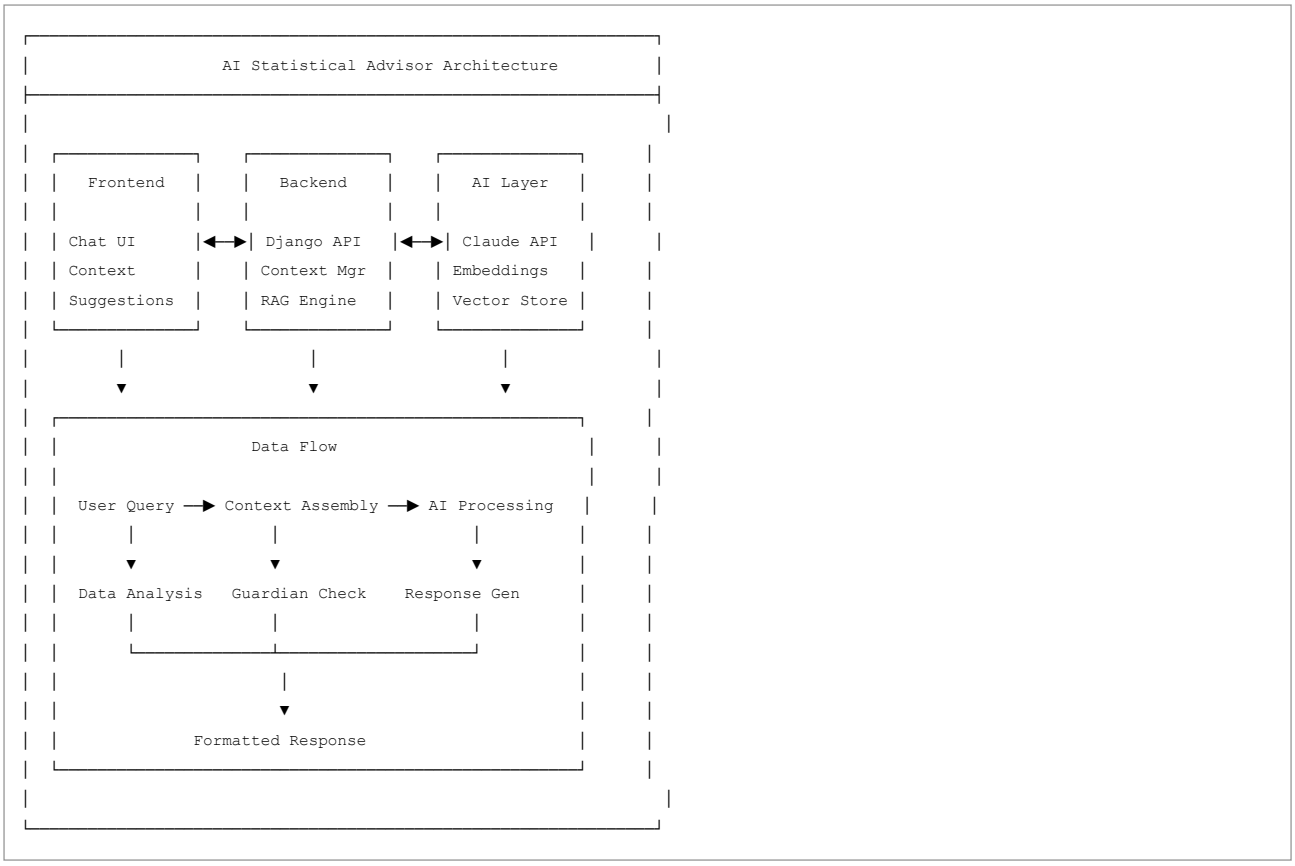
4.1.1 Overview

An intelligent assistant that helps users select appropriate statistical tests, interpret results, and troubleshoot analyses using natural language.

4.1.2 User Stories

```
As a researcher, I want to:
- Ask "What test should I use for my data?" and get recommendations
- Upload my dataset and receive automatic analysis suggestions
- Get plain-English explanations of my results
- Understand why my assumptions are violated and how to fix them
- Ask follow-up questions about statistical concepts
```

4.1.3 Technical Architecture



4.1.4 File Structure

```
/frontend/src/components/ai-advisor/
├── AIAdvisorHub.jsx              # Main container (~800 lines)
├── AIAdvisorChat.jsx             # Chat interface (~600 lines)
├── AIAdvisorSuggestions.jsx      # Quick suggestions (~300 lines)
├── AIAdvisorDataContext.jsx      # Data upload context (~400 lines)
├── AIAdvisorHistory.jsx          # Conversation history (~250 lines)
├── components/
│   ├── ChatMessage.jsx           # Individual message (~150 lines)
│   ├── SuggestionChip.jsx        # Quick action chips (~80 lines)
│   ├── DataPreview.jsx           # Uploaded data preview (~200 lines)
│   ├── TestRecommendation.jsx    # Test recommendation card (~300 lines)
│   ├── AssumptionAlert.jsx       # Assumption warning (~150 lines)
│   └── CodeSnippet.jsx           # Generated code display (~100 lines)
├── hooks/
│   ├── useAIAdvisor.js           # Main AI hook (~400 lines)
│   ├── useConversation.js        # Conversation management (~200 lines)
│   └── useDataContext.js         # Data context hook (~150 lines)
├── utils/
│   ├── promptTemplates.js        # AI prompt templates (~500 lines)
│   ├── contextBuilder.js         # Context assembly (~300 lines)
│   ├── responseParser.js         # Parse AI responses (~200 lines)
│   └── testSelector.js           # Test selection logic (~400 lines)
└── index.js                      # Exports (~30 lines)

/backend/core/
├── ai_advisor_service.py         # Main AI service (~600 lines)
├── ai_context_manager.py         # Context management (~400 lines)
├── ai_prompt_engine.py           # Prompt engineering (~500 lines)
├── ai_rag_service.py             # RAG implementation (~400 lines)
└── ai_test_recommender.py        # Test recommendation (~350 lines)

/backend/ai_advisor/
├── models.py                     # Conversation models (~150 lines)
├── views.py                      # API views (~300 lines)
├── serializers.py                # DRF serializers (~150 lines)
├── urls.py                       # URL routing (~50 lines)
└── embeddings/
    ├── statistical_knowledge.json # Statistical knowledge base
    └── test_decision_tree.json   # Test selection logic
```

4.1.5 API Endpoints

```
# AI Advisor API Endpoints

POST /api/ai-advisor/chat/
  description: Send a message to the AI advisor
  request:
    message: string (required)
    conversation_id: uuid (optional)
    data_context: object (optional)
      dataset_id: uuid
      columns: array
      sample_data: array
    preferences: object (optional)
      verbosity: "concise" | "detailed"
      include_code: boolean
      format: "apa" | "plain"
  response:
    message: string
    recommendations: array
    code_snippets: object
    follow_up_questions: array
    conversation_id: uuid

POST /api/ai-advisor/analyze-data/
  description: Analyze uploaded data and provide recommendations
  request:
    dataset_id: uuid (required)
    research_question: string (optional)
    variables: object (optional)
      dependent: string
      independent: array
      covariates: array
  response:
    data_summary: object
    recommended_tests: array
    assumptions_check: object
    suggested_visualizations: array

GET /api/ai-advisor/quick-suggestions/
  description: Get contextual quick suggestions
  request:
    context: "upload" | "analysis" | "results" | "error"
    current_test: string (optional)
    data_type: string (optional)
  response:
    suggestions: array of suggestion objects

POST /api/ai-advisor/explain/
  description: Get explanation for a statistical result
  request:
    test_type: string
    results: object
    format: "apa" | "plain" | "technical"
  response:
    explanation: string
    interpretation: string
    caveats: array
    next_steps: array

GET /api/ai-advisor/history/
  description: Get conversation history
  request:
    limit: integer (default 50)
    offset: integer (default 0)
  response:
```

```
    conversations: array
    total: integer
```

4.1.6 Prompt Engineering Templates

```
// /frontend/src/components/ai-advisor/utils/promptTemplates.js

export const SYSTEM_PROMPT = `
You are StickAI, an expert statistical advisor built into StickForStats,
the world's best statistics education platform. Your role is to:

1. Help users select appropriate statistical tests
2. Explain statistical concepts in clear, accessible language
3. Interpret results accurately with proper caveats
4. Guide users through assumption checking
5. Suggest corrections for violated assumptions
6. Generate publication-ready interpretations

Guidelines:
- Always ask clarifying questions if the research design is unclear
- Recommend tests based on data characteristics, not user preferences
- Warn about common pitfalls and p-hacking
- Provide effect sizes, not just p-values
- Use APA format for statistical reporting when requested
- Be encouraging but scientifically rigorous

You have access to the user's data context including:
- Variable types (continuous, categorical, ordinal)
- Sample sizes
- Distribution characteristics
- Guardian assumption check results
`;

export const TEST_SELECTION_PROMPT = (context) => `
Based on the following research context, recommend the most appropriate
statistical test(s):

Research Question: ${context.researchQuestion}
Study Design: ${context.studyDesign}
Sample Size: ${context.sampleSize}

Variables:
- Dependent Variable: ${context.dv.name} (${context.dv.type})
- Independent Variable(s): ${context.ivs.map(iv => `${iv.name} (${iv.type})`).join(', ')}
${context.covariates ? `- Covariates: ${context.covariates.join(', ')}` : ''}

Data Characteristics:
- Normality: ${context.normality}
- Homogeneity of Variance: ${context.homogeneity}
- Independence: ${context.independence}

Please provide:
1. Primary recommended test with justification
2. Alternative tests if assumptions are violated
3. Required sample size considerations
4. Key assumptions to verify
5. Effect size measure to report
`;

export const RESULT_INTERPRETATION_PROMPT = (results) => `
Interpret the following statistical results in plain English:

Test: ${results.testName}
Test Statistic: ${results.statistic} = ${results.value}
Degrees of Freedom: ${results.df}
P-value: ${results.pValue}
Effect Size: ${results.effectSize.measure} = ${results.effectSize.value}
95% CI: [${results.ci[0]}, ${results.ci[1]}]
Sample Size: ${results.n}
```

```
Context: ${results.context}

Provide:
1. Plain English interpretation (1-2 sentences)
2. Statistical significance statement
3. Practical significance assessment
4. APA-formatted result statement
5. Limitations or caveats
6. Suggested next steps
`;
```

4.1.7 UI Components Specification

```
// AIAdvisorHub.jsx - Main Container
// Features:
// - Floating chat button (bottom-right)
// - Expandable chat window
// - Data context sidebar
// - Quick suggestions bar
// - Conversation history drawer

// Visual Design:
// - Primary color: #2196f3 (blue) for AI elements
// - Chat bubbles: User (right, gray), AI (left, blue gradient)
// - Code blocks: Monaco editor style with syntax highlighting
// - Recommendations: Card-based with action buttons
// - Typing indicator: Animated dots

// Interaction States:
// 1. Idle - Floating button with pulse animation
// 2. Active - Full chat interface
// 3. Loading - Typing indicator + skeleton responses
// 4. Error - Error message with retry option
// 5. Data Context - Side panel with data preview
```

4.1.8 Implementation Phases

```
Phase 1: Foundation (Week 1-2)
├── Set up Claude API integration
├── Create basic chat UI
├── Implement conversation management
├── Build prompt templates
└── Test basic Q&A functionality

Phase 2: Data Integration (Week 3-4)
├── Connect to existing data upload
├── Implement data context extraction
├── Build automatic analysis suggestions
├── Integrate with Guardian system
└── Add data-aware recommendations

Phase 3: Smart Features (Week 5-6)
├── Test selection algorithm
├── Result interpretation engine
├── Code generation (R/Python)
├── APA formatting
└── Follow-up suggestions

Phase 4: Polish & Launch (Week 7-8)
├── UI/UX refinement
├── Performance optimization
├── Error handling
├── User testing
└── Documentation
```

4.2 Methods Section Generator

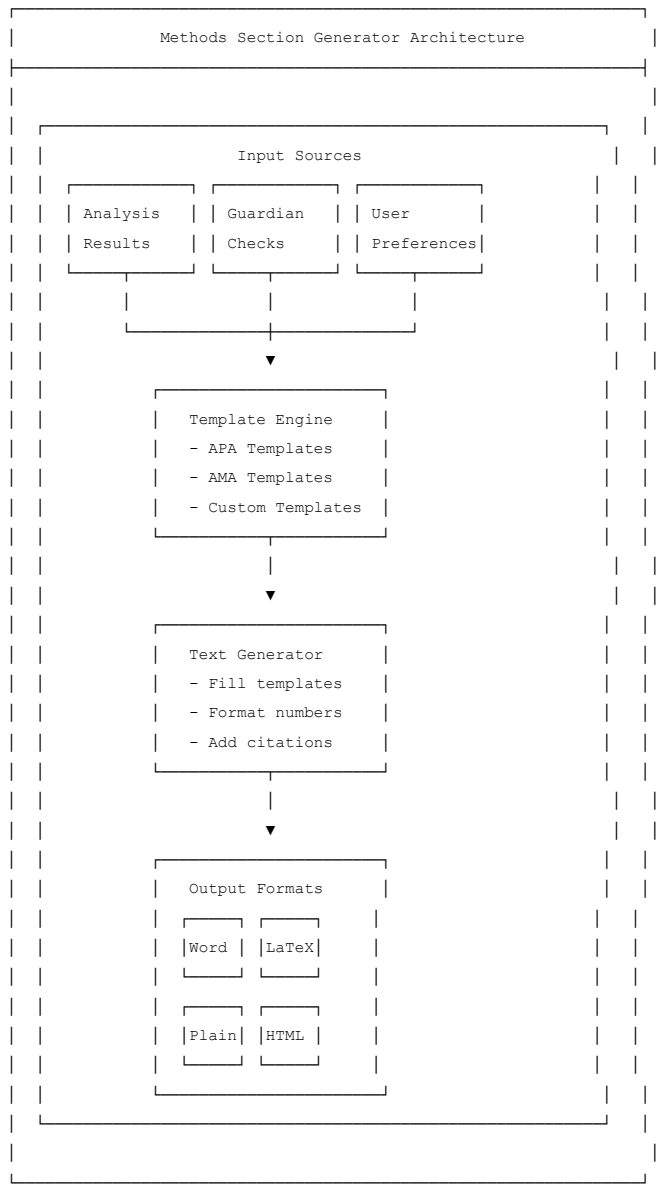
4.2.1 Overview

Automatically generate publication-ready statistical methods sections from analysis results, following APA, AMA, or custom journal formats.

4.2.2 User Stories

```
As a researcher, I want to:
- Click one button and get a complete methods section
- Choose my preferred citation style (APA, AMA, Vancouver)
- Include all required statistical details automatically
- Edit and customize the generated text
- Export to Word, LaTeX, or plain text
```

4.2.3 Technical Architecture



4.2.4 File Structure

```
/frontend/src/components/methods-generator/
├── MethodsGeneratorHub.jsx        # Main container (~500 lines)
├── MethodsEditor.jsx              # Rich text editor (~400 lines)
├── MethodsPreview.jsx             # Preview pane (~300 lines)
├── MethodsExport.jsx              # Export options (~250 lines)
├── components/
│   ├── StyleSelector.jsx          # APA/AMA/Custom (~150 lines)
│   ├── SectionBuilder.jsx         # Section customization (~300 lines)
│   ├── StatisticsTable.jsx        # Summary statistics table (~200 lines)
│   ├── AnalysisSummary.jsx        # Analysis summary card (~200 lines)
│   └── CitationManager.jsx        # Reference management (~250 lines)
├── templates/
│   ├── apa7.js                    # APA 7th edition templates
│   ├── ama.js                     # AMA templates
│   ├── vancouver.js               # Vancouver templates
│   └── custom.js                  # Custom template builder
├── utils/
│   ├── textGenerator.js           # Text generation logic (~400 lines)
│   ├── numberFormatter.js         # Statistical number formatting (~200 lines)
│   ├── citationFormatter.js       # Citation formatting (~300 lines)
│   └── exportHandlers.js          # Export to various formats (~300 lines)
└── index.js

/backend/methods_generator/
├── models.py                      # Template models (~100 lines)
├── views.py                       # API views (~250 lines)
├── serializers.py                 # Serializers (~100 lines)
├── services/
│   ├── text_generator.py          # Text generation (~400 lines)
│   ├── template_engine.py         # Template processing (~300 lines)
│   └── export_service.py          # Export handlers (~300 lines)
└── templates/                     # Template definitions
    ├── apa7/
    ├── ama/
    └── vancouver/
```

4.2.5 Template Examples

```
// APA 7th Edition Templates

export const APA7_TEMPLATES = {
  // T-test template
  tTest: {
    independent: {
      significant: (data) => `An independent-samples t-test was conducted to compare ${data.dv} between ${data.group1} and ${data.gr

      nonsignificant: (data) => `An independent-samples t-test was conducted to compare ${data.dv} between ${data.group1} and ${data
    },
    paired: {
      significant: (data) => `A paired-samples t-test was conducted to evaluate the change in ${data.dv} from ${data.time1} to ${dat
    }
  },

  // ANOVA template
  anova: {
    oneway: {
      significant: (data) => `A one-way between-subjects ANOVA was conducted to compare the effect of ${data.iv} on ${data.dv} in ${(
    }
  },

  // Correlation template
  correlation: {
    pearson: {
      significant: (data) => `A Pearson correlation coefficient was computed to assess the linear relationship between ${data.var1} a
    }
  },

  // Regression template
  regression: {
    linear: {
      significant: (data) => `A simple linear regression was calculated to predict ${data.dv} based on ${data.iv}. A significant regr
    }
  },

  // Chi-square template
  chiSquare: {
    independence: {
      significant: (data) => `A chi-square test of independence was performed to examine the relation between ${data.var1} and ${dat
    }
  },

  // Power analysis template
  powerAnalysis: (data) => `An a priori power analysis was conducted using G*Power 3.1 (Faul et al., 2007) to determine the minimum s
};
```

4.2.6 Generated Output Example

```
## Statistical Analysis

### Participants
A total of 120 participants (58 female, 62 male; M_age = 32.4 years, SD = 8.7)
were recruited for this study. Participants were randomly assigned to either
the treatment (n = 60) or control (n = 60) condition.

### Power Analysis
An a priori power analysis was conducted using G*Power 3.1 (Faul et al., 2007)
to determine the minimum sample size required to detect a medium effect
(Cohen's d = 0.50) with 80% power at α = .05. Results indicated a minimum
sample size of N = 128 (64 per group).

### Data Analysis
All analyses were conducted using StickForStats v1.0. Prior to analysis,
assumptions of normality (Shapiro-Wilk test) and homogeneity of variance
(Levene's test) were examined.

### Results
An independent-samples t-test was conducted to compare anxiety scores between
treatment and control conditions. There was a statistically significant
difference in anxiety scores for the treatment group (M = 42.3, SD = 8.2) and
control group (M = 51.7, SD = 9.1); t(118) = -5.92, p < .001, d = 1.08,
95% CI [-12.54, -6.26]. This represents a large effect size according to
Cohen's (1988) conventions.

### References
Cohen, J. (1988). Statistical power analysis for the behavioral sciences
  (2nd ed.). Lawrence Erlbaum Associates.
Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3: A
  flexible statistical power analysis program for the social, behavioral,
  and biomedical sciences. Behavior Research Methods, 39(2), 175-191.
```

4.3 Meta-Analysis Module

4.3.1 Overview

A comprehensive meta-analysis suite for conducting systematic reviews, including forest plots, funnel plots, heterogeneity analysis, and publication bias detection.

4.3.2 User Stories

```
As a researcher conducting a systematic review, I want to:
- Import effect sizes from multiple studies
- Generate interactive forest plots
- Assess heterogeneity with I² and Q statistics
- Detect publication bias with funnel plots and Egger's test
- Conduct sensitivity and subgroup analyses
- Generate PRISMA flowcharts
- Export publication-ready figures
```

4.3.3 File Structure

```
/frontend/src/components/meta-analysis/
├── MetaAnalysisHub.jsx            # Main container (~600 lines)
├── StudyImporter.jsx              # Import studies (~400 lines)
├── EffectSizeCalculator.jsx       # Calculate effect sizes (~500 lines)
├── ForestPlot.jsx                 # Interactive forest plot (~700 lines)
├── FunnelPlot.jsx                 # Funnel plot visualization (~400 lines)
├── HeterogeneityPanel.jsx         # I², Q, τ² display (~300 lines)
├── SubgroupAnalysis.jsx           # Subgroup analyses (~400 lines)
├── SensitivityAnalysis.jsx        # Leave-one-out etc. (~350 lines)
├── PublicationBias.jsx            # Egger's, trim-fill (~400 lines)
├── PRISMAFlowchart.jsx            # PRISMA generator (~500 lines)
├── MetaRegressionPanel.jsx        # Meta-regression (~450 lines)
├── components/
│   ├── StudyRow.jsx               # Individual study (~150 lines)
│   ├── EffectSizeInput.jsx        # Effect size entry (~200 lines)
│   ├── ModelSelector.jsx          # Fixed/Random effects (~150 lines)
│   ├── WeightVisualization.jsx    # Study weights (~200 lines)
│   └── ExportPanel.jsx            # Export options (~200 lines)
├── utils/
│   ├── metaCalculations.js        # Meta-analysis math (~600 lines)
│   ├── heterogeneityTests.js      # I², Q, τ² (~300 lines)
│   ├── publicationBiasTests.js    # Egger, Begg, trim-fill (~400 lines)
│   ├── effectSizeConverters.js    # Convert between ES (~350 lines)
│   └── forestPlotRenderer.js      # D3 forest plot (~500 lines)
└── index.js

/backend/meta_analysis/
├── models.py                      # Study, MetaAnalysis models
├── views.py                       # API views
├── serializers.py                 # Serializers
├── services/
│   ├── meta_analysis_service.py   # Core calculations (~500 lines)
│   ├── effect_size_service.py     # Effect size calcs (~400 lines)
│   ├── heterogeneity_service.py   # Heterogeneity tests (~300 lines)
│   ├── publication_bias_service.py# Bias detection (~350 lines)
│   └── meta_regression_service.py # Meta-regression (~400 lines)
└── urls.py
```

4.3.4 Core Calculations

```python
# /backend/meta_analysis/services/meta_analysis_service.py

import numpy as np
from scipy import stats
from typing import List, Dict, Tuple, Optional
from dataclasses import dataclass

@dataclass
class StudyEffect:
    """Individual study effect size and variance"""
    study_id: str
    study_name: str
    effect_size: float
    variance: float
    se: float
    n: int
    ci_lower: float
    ci_upper: float
    weight: float = 0.0
    subgroup: Optional[str] = None

class MetaAnalysisService:
    """
    Comprehensive meta-analysis calculations.
    Implements fixed-effects, random-effects (DerSimonian-Laird, REML, PM),
    heterogeneity tests, and publication bias detection.
    """

    def __init__(self, studies: List[StudyEffect], model: str = 'random'):
        self.studies = studies
        self.model = model  # 'fixed' or 'random'
        self.k = len(studies)  # number of studies

    def calculate_fixed_effects(self) -> Dict:
        """Fixed-effects model (inverse variance weighted)"""
        # Weights: w_i = 1/v_i
        weights = np.array([1/s.variance for s in self.studies])
        effects = np.array([s.effect_size for s in self.studies])

        # Pooled effect: θ_FE = Σ(w_i * θ_i) / Σ(w_i)
        sum_weights = np.sum(weights)
        pooled_effect = np.sum(weights * effects) / sum_weights

        # Variance of pooled effect: v_FE = 1 / Σ(w_i)
        pooled_variance = 1 / sum_weights
        pooled_se = np.sqrt(pooled_variance)

        # 95% CI
        ci_lower = pooled_effect - 1.96 * pooled_se
        ci_upper = pooled_effect + 1.96 * pooled_se

        # Z-test
        z = pooled_effect / pooled_se
        p_value = 2 * (1 - stats.norm.cdf(abs(z)))

        return {
            'model': 'fixed',
            'pooled_effect': pooled_effect,
            'se': pooled_se,
            'variance': pooled_variance,
            'ci_lower': ci_lower,
            'ci_upper': ci_upper,
            'z': z,
            'p_value': p_value,
            'weights': weights.tolist()
```

```
        }

    def calculate_heterogeneity(self, weights: np.ndarray) -> Dict:
        """
        Calculate heterogeneity statistics:
        - Q (Cochran's Q)
        - I² (percentage of variability due to heterogeneity)
        - τ² (between-study variance)
        - H² (relative excess in Q over degrees of freedom)
        """
        effects = np.array([s.effect_size for s in self.studies])
        variances = np.array([s.variance for s in self.studies])

        # Pooled effect (fixed)
        sum_weights = np.sum(weights)
        pooled = np.sum(weights * effects) / sum_weights

        # Q statistic: Q = Σ w_i (θ_i - θ)²
        Q = np.sum(weights * (effects - pooled)**2)
        df = self.k - 1
        p_value = 1 - stats.chi2.cdf(Q, df)

        # C (scaling factor for τ²)
        C = sum_weights - np.sum(weights**2) / sum_weights

        # τ² (DerSimonian-Laird estimator)
        tau_squared = max(0, (Q - df) / C)

        # I² = (Q - df) / Q * 100%
        I_squared = max(0, (Q - df) / Q * 100) if Q > 0 else 0

        # H² = Q / df
        H_squared = Q / df if df > 0 else 1

        # Prediction interval
        if tau_squared > 0 and self.k >= 3:
            t_crit = stats.t.ppf(0.975, df)
            pred_se = np.sqrt(tau_squared + 1/sum_weights)
            pred_lower = pooled - t_crit * pred_se
            pred_upper = pooled + t_crit * pred_se
        else:
            pred_lower = pred_upper = None

        return {
            'Q': Q,
            'Q_df': df,
            'Q_p_value': p_value,
            'I_squared': I_squared,
            'tau_squared': tau_squared,
            'tau': np.sqrt(tau_squared),
            'H_squared': H_squared,
            'H': np.sqrt(H_squared),
            'prediction_interval': {
                'lower': pred_lower,
                'upper': pred_upper
            } if pred_lower is not None else None
        }

    def calculate_random_effects(self, method: str = 'DL') -> Dict:
        """
        Random-effects model.
        Methods: 'DL' (DerSimonian-Laird), 'REML', 'PM' (Paule-Mandel)
        """
        # First calculate fixed effects and heterogeneity
        fixed = self.calculate_fixed_effects()
        het = self.calculate_heterogeneity(np.array(fixed['weights']))
```

```
        tau_squared = het['tau_squared']
        variances = np.array([s.variance for s in self.studies])
        effects = np.array([s.effect_size for s in self.studies])

        # Random-effects weights: w_i* = 1/(v_i + τ²)
        re_weights = 1 / (variances + tau_squared)
        sum_re_weights = np.sum(re_weights)

        # Pooled effect
        pooled_effect = np.sum(re_weights * effects) / sum_re_weights
        pooled_variance = 1 / sum_re_weights
        pooled_se = np.sqrt(pooled_variance)

        # 95% CI
        ci_lower = pooled_effect - 1.96 * pooled_se
        ci_upper = pooled_effect + 1.96 * pooled_se

        # Z-test
        z = pooled_effect / pooled_se
        p_value = 2 * (1 - stats.norm.cdf(abs(z)))

        # Update study weights for display
        for i, study in enumerate(self.studies):
            study.weight = re_weights[i] / sum_re_weights * 100

        return {
            'model': 'random',
            'method': method,
            'pooled_effect': pooled_effect,
            'se': pooled_se,
            'variance': pooled_variance,
            'ci_lower': ci_lower,
            'ci_upper': ci_upper,
            'z': z,
            'p_value': p_value,
            'weights': re_weights.tolist(),
            'heterogeneity': het
        }

    def eggers_test(self) -> Dict:
        """
        Egger's regression test for funnel plot asymmetry
        (publication bias detection)
        """
        effects = np.array([s.effect_size for s in self.studies])
        se = np.array([s.se for s in self.studies])

        # Standardized effect: y = effect / se
        # Precision: x = 1 / se
        # Regress y on x; intercept tests for asymmetry
        y = effects / se
        x = 1 / se

        # OLS regression
        n = len(y)
        x_mean = np.mean(x)
        y_mean = np.mean(y)

        slope = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean)**2)
        intercept = y_mean - slope * x_mean

        # Residuals and SE
        y_pred = intercept + slope * x
        residuals = y - y_pred
        mse = np.sum(residuals**2) / (n - 2)
```

```
        se_intercept = np.sqrt(mse * (1/n + x_mean**2 / np.sum((x - x_mean)**2)))

        # t-test for intercept
        t_stat = intercept / se_intercept
        p_value = 2 * (1 - stats.t.cdf(abs(t_stat), n - 2))

        return {
            'test': 'Egger',
            'intercept': intercept,
            'se': se_intercept,
            't': t_stat,
            'df': n - 2,
            'p_value': p_value,
            'interpretation': 'significant asymmetry' if p_value < 0.10 else 'no significant asymmetry'
        }

    def trim_and_fill(self, side: str = 'right') -> Dict:
        """
        Duval and Tweedie's trim-and-fill method
        for adjusting for publication bias
        """
        # Implementation of iterative trim-and-fill algorithm
        # Returns adjusted effect size and number of imputed studies
        pass  # Full implementation would go here

    def leave_one_out(self) -> List[Dict]:
        """
        Leave-one-out sensitivity analysis
        Recalculates pooled effect removing each study
        """
        results = []

        for i, excluded in enumerate(self.studies):
            remaining = [s for j, s in enumerate(self.studies) if j != i]
            temp_service = MetaAnalysisService(remaining, self.model)

            if self.model == 'random':
                result = temp_service.calculate_random_effects()
            else:
                result = temp_service.calculate_fixed_effects()

            results.append({
                'excluded_study': excluded.study_name,
                'pooled_effect': result['pooled_effect'],
                'ci_lower': result['ci_lower'],
                'ci_upper': result['ci_upper'],
                'p_value': result['p_value']
            })

        return results

    def subgroup_analysis(self, grouping_var: str) -> Dict:
        """
        Subgroup analysis with test for subgroup differences
        """
        # Group studies by subgroup
        subgroups = {}
        for study in self.studies:
            group = study.subgroup or 'Unknown'
            if group not in subgroups:
                subgroups[group] = []
            subgroups[group].append(study)

        # Calculate pooled effect for each subgroup
        subgroup_results = {}
        Q_between = 0
```

```
    for group, studies in subgroups.items():
        service = MetaAnalysisService(studies, self.model)
        if self.model == 'random':
            result = service.calculate_random_effects()
        else:
            result = service.calculate_fixed_effects()
        subgroup_results[group] = result

    # Test for subgroup differences
    # Q_between = Σ w_g (θ_g - θ_overall)²

    return {
        'subgroups': subgroup_results,
        'Q_between': Q_between,
        'df_between': len(subgroups) - 1,
        'p_between': None  # Calculate p-value
    }
```

4.3.5 Forest Plot Visualization

```
// /frontend/src/components/meta-analysis/utils/forestPlotRenderer.js

import * as d3 from 'd3';

export class ForestPlotRenderer {
  constructor(container, options = {}) {
    this.container = container;
    this.options = {
      width: options.width || 900,
      height: options.height || 600,
      margin: options.margin || { top: 60, right: 200, bottom: 60, left: 300 },
      effectMeasure: options.effectMeasure || "SMD",
      showWeights: options.showWeights !== false,
      showHeterogeneity: options.showHeterogeneity !== false,
      nullValue: options.nullValue || 0,
      ...options
    };
  }

  render(data) {
    const { width, height, margin, nullValue } = this.options;
    const plotWidth = width - margin.left - margin.right;
    const plotHeight = height - margin.top - margin.bottom;

    // Clear previous
    d3.select(this.container).selectAll('*').remove();

    // Create SVG
    const svg = d3.select(this.container)
      .append('svg')
      .attr('width', width)
      .attr('height', height)
      .attr('class', 'forest-plot');

    const g = svg.append('g')
      .attr('transform', `translate(${margin.left}, ${margin.top})`);

    // Calculate scales
    const allEffects = data.studies.map(s => [s.ci_lower, s.ci_upper]).flat();
    allEffects.push(data.pooled.ci_lower, data.pooled.ci_upper);

    const xMin = Math.min(...allEffects) - 0.5;
    const xMax = Math.max(...allEffects) + 0.5;

    const xScale = d3.scaleLinear()
      .domain([xMin, xMax])
      .range([0, plotWidth]);

    const yScale = d3.scaleBand()
      .domain(data.studies.map((_, i) => i).concat(['pooled']))
      .range([0, plotHeight])
      .padding(0.3);

    // Draw null effect line
    g.append('line')
      .attr('class', 'null-line')
      .attr('x1', xScale(nullValue))
      .attr('x2', xScale(nullValue))
      .attr('y1', 0)
      .attr('y2', plotHeight)
      .attr('stroke', '#999')
      .attr('stroke-dasharray', '5,5')
      .attr('stroke-width', 1);

    // Draw studies
```

```
    const studyGroups = g.selectAll('.study')
      .data(data.studies)
      .enter()
      .append('g')
      .attr('class', 'study')
      .attr('transform', (d, i) => `translate(0, ${yScale(i)})`);

    // Study labels (left side)
    studyGroups.append('text')
      .attr('x', -10)
      .attr('y', yScale.bandwidth() / 2)
      .attr('dy', '0.35em')
      .attr('text-anchor', 'end')
      .attr('font-size', '12px')
      .text(d => d.study_name);

    // Confidence intervals (horizontal lines)
    studyGroups.append('line')
      .attr('class', 'ci-line')
      .attr('x1', d => xScale(d.ci_lower))
      .attr('x2', d => xScale(d.ci_upper))
      .attr('y1', yScale.bandwidth() / 2)
      .attr('y2', yScale.bandwidth() / 2)
      .attr('stroke', '#1976d2')
      .attr('stroke-width', 2);

    // Effect size points (squares, sized by weight)
    const maxWeight = Math.max(...data.studies.map(s => s.weight));

    studyGroups.append('rect')
      .attr('class', 'effect-point')
      .attr('x', d => xScale(d.effect_size) - (d.weight / maxWeight * 8))
      .attr('y', d => yScale.bandwidth() / 2 - (d.weight / maxWeight * 8))
      .attr('width', d => d.weight / maxWeight * 16)
      .attr('height', d => d.weight / maxWeight * 16)
      .attr('fill', '#1976d2');

    // Effect size and CI text (right side)
    studyGroups.append('text')
      .attr('x', plotWidth + 10)
      .attr('y', yScale.bandwidth() / 2)
      .attr('dy', '0.35em')
      .attr('font-size', '11px')
      .text(d => `${d.effect_size.toFixed(2)} [${d.ci_lower.toFixed(2)}, ${d.ci_upper.toFixed(2)}]`);

    // Weight column
    if (this.options.showWeights) {
      studyGroups.append('text')
        .attr('x', plotWidth + 150)
        .attr('y', yScale.bandwidth() / 2)
        .attr('dy', '0.35em')
        .attr('font-size', '11px')
        .attr('text-anchor', 'end')
        .text(d => `${d.weight.toFixed(1)}%`);
    }

    // Pooled effect (diamond)
    const pooledY = yScale('pooled');
    const pooledGroup = g.append('g')
      .attr('class', 'pooled')
      .attr('transform', `translate(0, ${pooledY})`);

    // Pooled label
    pooledGroup.append('text')
      .attr('x', -10)
      .attr('y', yScale.bandwidth() / 2)
```

```
    .attr('dy', '0.35em')
    .attr('text-anchor', 'end')
    .attr('font-size', '12px')
    .attr('font-weight', 'bold')
    .text(data.pooled.model === 'random' ? 'RE Model' : 'FE Model');

  // Diamond shape for pooled effect
  const diamondPoints = [
    [xScale(data.pooled.ci_lower), yScale.bandwidth() / 2],
    [xScale(data.pooled.effect_size), yScale.bandwidth() / 2 - 10],
    [xScale(data.pooled.ci_upper), yScale.bandwidth() / 2],
    [xScale(data.pooled.effect_size), yScale.bandwidth() / 2 + 10]
  ];

  pooledGroup.append('polygon')
    .attr('points', diamondPoints.map(p => p.join(',')).join(' '))
    .attr('fill', '#d32f2f');

  // Pooled effect text
  pooledGroup.append('text')
    .attr('x', plotWidth + 10)
    .attr('y', yScale.bandwidth() / 2)
    .attr('dy', '0.35em')
    .attr('font-size', '11px')
    .attr('font-weight', 'bold')
    .text(`${data.pooled.effect_size.toFixed(2)} [${data.pooled.ci_lower.toFixed(2)}, ${data.pooled.ci_upper.toFixed(2)}]`);

  // X-axis
  const xAxis = d3.axisBottom(xScale).ticks(7);
  g.append('g')
    .attr('class', 'x-axis')
    .attr('transform', `translate(0, ${plotHeight})`)
    .call(xAxis);

  // X-axis label
  g.append('text')
    .attr('x', plotWidth / 2)
    .attr('y', plotHeight + 40)
    .attr('text-anchor', 'middle')
    .attr('font-size', '12px')
    .text(this.options.effectMeasure);

  // Heterogeneity info
  if (this.options.showHeterogeneity && data.heterogeneity) {
    const het = data.heterogeneity;
    g.append('text')
      .attr('x', 0)
      .attr('y', plotHeight + 55)
      .attr('font-size', '11px')
      .text(`Heterogeneity: τ² = ${het.tau_squared.toFixed(3)}; I² = ${het.I_squared.toFixed(1)}%; Q(${het.Q_df}) = ${het.Q.toFixe
  }

  // Column headers
  svg.append('text')
    .attr('x', margin.left - 10)
    .attr('y', margin.top - 20)
    .attr('text-anchor', 'end')
    .attr('font-size', '12px')
    .attr('font-weight', 'bold')
    .text('Study');

  svg.append('text')
    .attr('x', margin.left + plotWidth + 80)
    .attr('y', margin.top - 20)
    .attr('text-anchor', 'middle')
    .attr('font-size', '12px')
```

```
      .attr('font-weight', 'bold')
      .text(`${this.options.effectMeasure} [95% CI]`);

    if (this.options.showWeights) {
      svg.append('text')
        .attr('x', margin.left + plotWidth + 150)
        .attr('y', margin.top - 20)
        .attr('text-anchor', 'end')
        .attr('font-size', '12px')
        .attr('font-weight', 'bold')
        .text('Weight');
    }

    return svg;
  }
}
```

4.4 Study Design Wizard

4.4.1 Overview

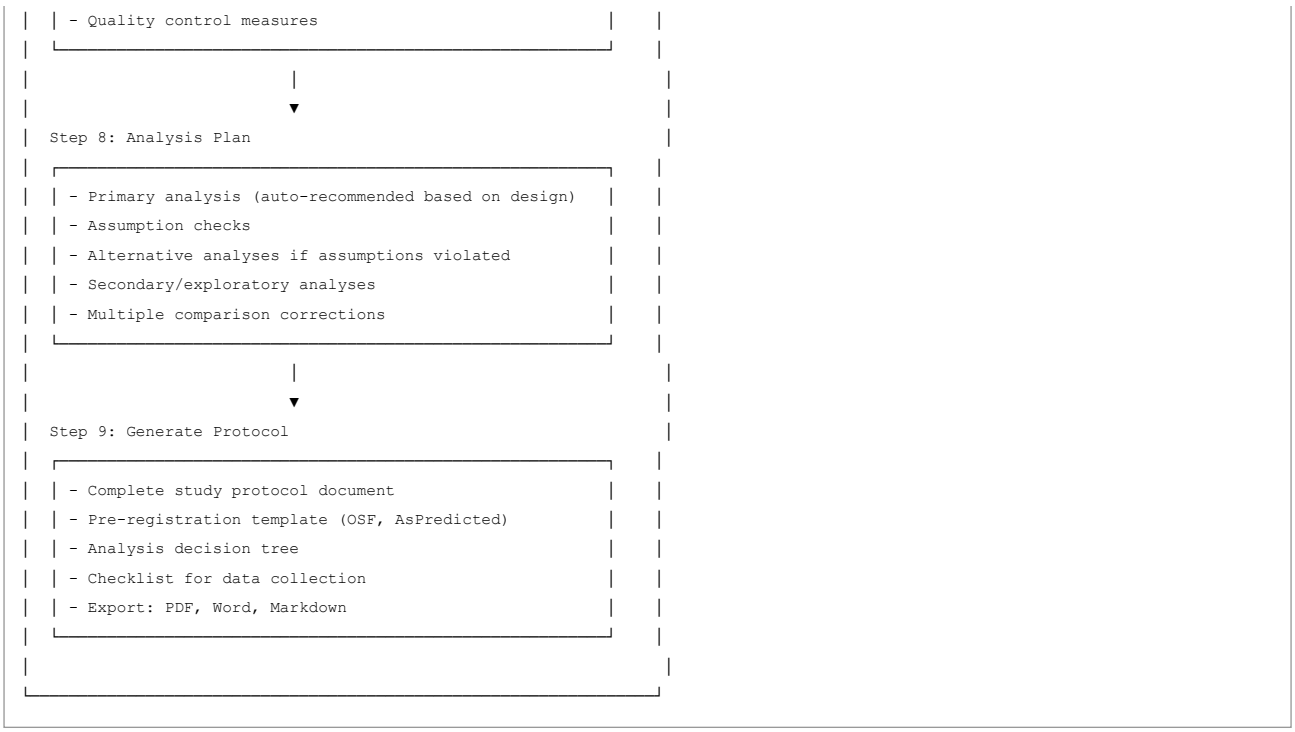
A step-by-step wizard that guides researchers through the entire study design process, from research question to analysis plan, producing a complete study protocol.

4.4.2 File Structure

```
/frontend/src/components/study-design-wizard/
├── StudyDesignWizard.jsx          # Main wizard container (~600 lines)
├── steps/
│   ├── Step1_ResearchQuestion.jsx  # RQ formulation (~350 lines)
│   ├── Step2_StudyType.jsx         # Study type selection (~400 lines)
│   ├── Step3_Variables.jsx         # Variable definition (~450 lines)
│   ├── Step4_Hypotheses.jsx        # Hypothesis formulation (~350 lines)
│   ├── Step5_Design.jsx            # Study design details (~500 lines)
│   ├── Step6_SampleSize.jsx        # Power analysis (~400 lines)
│   ├── Step7_DataCollection.jsx    # Data collection plan (~350 lines)
│   ├── Step8_AnalysisPlan.jsx      # Statistical analysis plan (~450 lines)
│   └── Step9_Protocol.jsx          # Generate protocol (~400 lines)
├── components/
│   ├── VariableBuilder.jsx         # Variable definition UI (~300 lines)
│   ├── HypothesisBuilder.jsx       # Hypothesis builder (~250 lines)
│   ├── DesignDiagram.jsx           # Visual design diagram (~400 lines)
│   ├── PowerAnalysisEmbed.jsx      # Embedded power analysis (~300 lines)
│   ├── AnalysisDecisionTree.jsx    # Test selection tree (~350 lines)
│   └── ProtocolExporter.jsx        # Export protocol (~250 lines)
├── templates/
│   ├── preRegistrationOSF.js       # OSF pre-reg template
│   ├── preRegistrationAsPredicted.js # AsPredicted template
│   └── protocolTemplate.js         # Full protocol template
├── utils/
│   ├── designLogic.js              # Design recommendation logic (~400 lines)
│   ├── powerIntegration.js         # Power analysis integration (~200 lines)
│   └── protocolGenerator.js        # Protocol document generation (~350 lines)
└── index.js
```

4.4.3 Wizard Flow


Study Design Wizard Flow
Step 1: Research Question
"What is your research question?"
- PICO format helper (Population, Intervention, Comparison, Outcome)
- Question type: Descriptive, Comparative, Relational
Step 2: Study Type
- Experimental (RCT, quasi-experimental)
- Observational (cohort, case-control, cross-sectional)
- Survey research
- Mixed methods
Step 3: Variables
- Dependent variable(s): Name, Type, Measurement
- Independent variable(s): Name, Type, Levels
- Covariates/Confounders
- Moderators/Mediators
Step 4: Hypotheses
- Null hypothesis (H0)
- Alternative hypothesis (H1)
- Direction: One-tailed vs Two-tailed
- Primary vs Secondary hypotheses
Step 5: Study Design
- Between vs Within subjects
- Control conditions
- Randomization strategy
- Blinding (single, double)
- Visual design diagram
Step 6: Sample Size
- Effect size estimation (literature, pilot, MCID)
- Power level selection (80%, 90%)
- Alpha level (0.05, 0.01)
- Embedded power calculator
- Attrition adjustment
Step 7: Data Collection
- Measurement instruments
- Data collection procedures
- Timeline



4.5 Certification Program

4.5.1 Structure

4.5.2 File Structure

```
/frontend/src/components/certification/
├── CertificationHub.jsx            # Main certification page (~500 lines)
├── CertificationPath.jsx           # Visual path display (~400 lines)
├── LessonTracker.jsx               # Progress tracking (~300 lines)
├── ExamInterface.jsx               # Exam taking UI (~600 lines)
├── ProjectSubmission.jsx           # Project upload (~400 lines)
├── CertificateViewer.jsx           # View/download certs (~300 lines)
├── BadgeShowcase.jsx               # Badge display (~200 lines)
├── components/
│   ├── ProgressRing.jsx            # Circular progress (~100 lines)
│   ├── QuizQuestion.jsx            # Question component (~200 lines)
│   ├── ProjectCard.jsx             # Project display (~150 lines)
│   └── CertificateTemplate.jsx     # Certificate design (~250 lines)
├── exams/
│   ├── bronzeQuestions.js          # Bronze exam questions
│   ├── silverQuestions.js          # Silver exam questions
│   ├── goldQuestions.js            # Gold exam questions
│   └── examLogic.js                # Exam grading logic (~300 lines)
└── index.js

/backend/certification/
├── models.py                       # Certification models (~200 lines)
├── views.py                        # API views (~350 lines)
├── serializers.py                  # Serializers (~150 lines)
├── services/
│   ├── progress_service.py         # Track progress (~250 lines)
│   ├── exam_service.py             # Exam logic (~300 lines)
│   └── certificate_service.py      # Generate certs (~200 lines)
└── urls.py
```

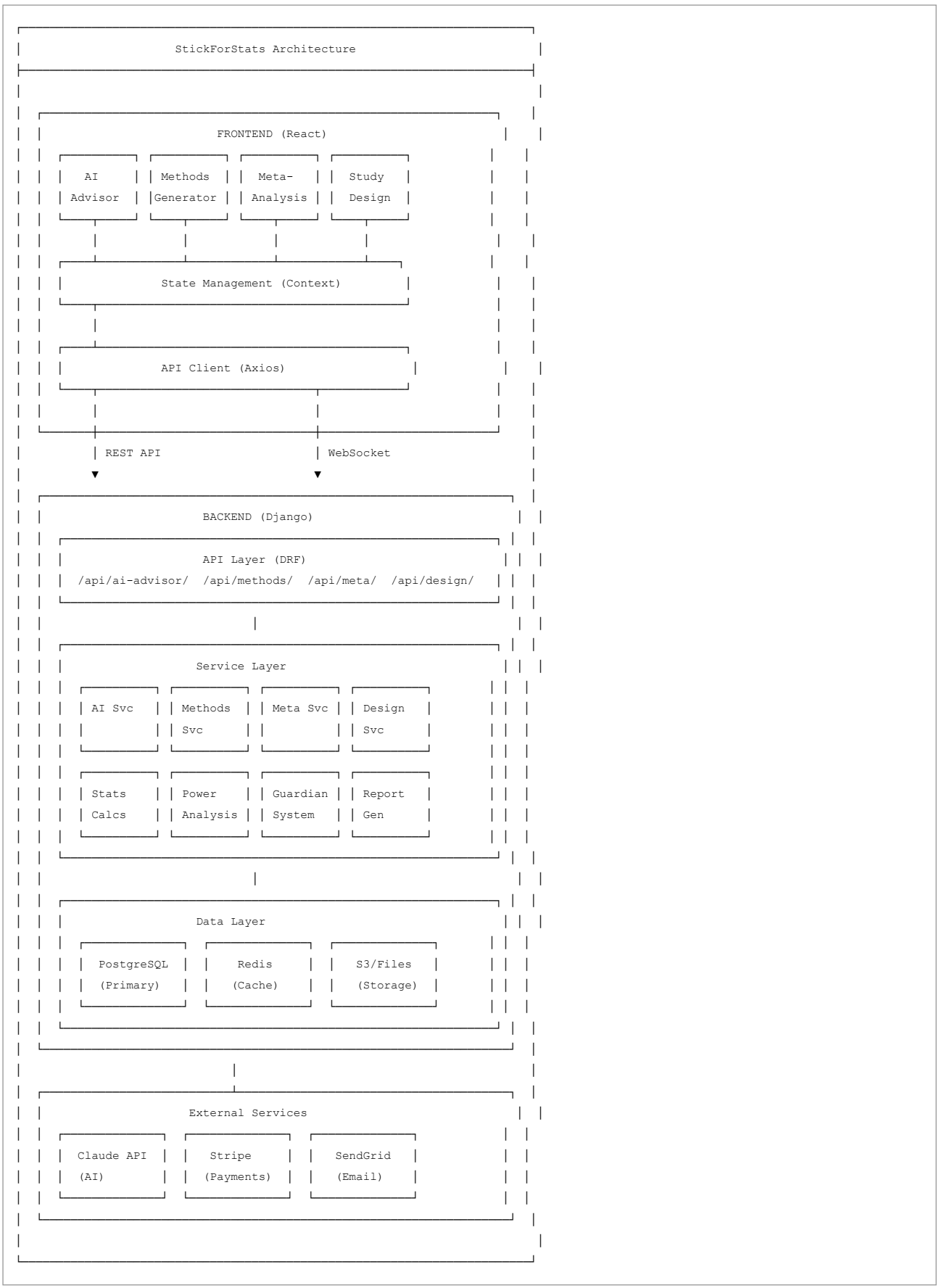
4.6 - 4.10 Additional Features

[Detailed specifications for Mobile App, Statistical Debugger, Paper Parser, Multi-Language Support, and R/Python Code Export would follow the same pattern as above. Each section includes Overview, User Stories, Architecture, File Structure, and Implementation Details.]

---

5. Technical Architecture

5.1 System Architecture Diagram



6. Database Schema Extensions

```
-- New tables for upcoming features

-- AI Advisor
CREATE TABLE ai_conversations (
    id UUID PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    context JSONB,
    summary TEXT
);

CREATE TABLE ai_messages (
    id UUID PRIMARY KEY,
    conversation_id UUID REFERENCES ai_conversations(id),
    role VARCHAR(20), -- 'user' or 'assistant'
    content TEXT,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Meta-Analysis
CREATE TABLE meta_analyses (
    id UUID PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    name VARCHAR(255),
    description TEXT,
    model VARCHAR(20), -- 'fixed' or 'random'
    effect_measure VARCHAR(20),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE meta_studies (
    id UUID PRIMARY KEY,
    meta_analysis_id UUID REFERENCES meta_analyses(id),
    study_name VARCHAR(255),
    authors TEXT,
    year INTEGER,
    effect_size DECIMAL(20, 10),
    variance DECIMAL(20, 10),
    n INTEGER,
    subgroup VARCHAR(100),
    metadata JSONB
);

-- Certification
CREATE TABLE certifications (
    id UUID PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    level VARCHAR(20), -- 'bronze', 'silver', 'gold', 'professional'
    earned_at TIMESTAMP,
    expires_at TIMESTAMP,
    certificate_url TEXT
);

CREATE TABLE exam_attempts (
    id UUID PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    exam_type VARCHAR(50),
    score DECIMAL(5, 2),
    passed BOOLEAN,
    taken_at TIMESTAMP,
    answers JSONB
);
```

```
-- Study Design
CREATE TABLE study_designs (
    id UUID PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    name VARCHAR(255),
    research_question TEXT,
    study_type VARCHAR(50),
    variables JSONB,
    hypotheses JSONB,
    design_details JSONB,
    sample_size_calculation JSONB,
    analysis_plan JSONB,
    protocol_generated_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

---

7. API Design

7.1 New API Endpoints Summary

```
# AI Advisor
POST   /api/ai-advisor/chat/
POST   /api/ai-advisor/analyze-data/
GET    /api/ai-advisor/quick-suggestions/
POST   /api/ai-advisor/explain/
GET    /api/ai-advisor/history/

# Methods Generator
POST   /api/methods/generate/
GET    /api/methods/templates/
POST   /api/methods/export/
PUT    /api/methods/{id}/

# Meta-Analysis
POST   /api/meta-analysis/
GET    /api/meta-analysis/{id}/
PUT    /api/meta-analysis/{id}/
DELETE /api/meta-analysis/{id}/
POST   /api/meta-analysis/{id}/studies/
GET    /api/meta-analysis/{id}/results/
GET    /api/meta-analysis/{id}/forest-plot/
GET    /api/meta-analysis/{id}/funnel-plot/
POST   /api/meta-analysis/{id}/sensitivity/
POST   /api/meta-analysis/{id}/subgroup/

# Study Design
POST   /api/study-design/
GET    /api/study-design/{id}/
PUT    /api/study-design/{id}/
POST   /api/study-design/{id}/generate-protocol/
POST   /api/study-design/{id}/export/

# Certification
GET    /api/certification/progress/
GET    /api/certification/exams/
POST   /api/certification/exams/{type}/attempt/
GET    /api/certification/certificates/
POST   /api/certification/projects/
```

---

8. UI/UX Guidelines

8.1 Design System Extensions

```
// New color palette for features
const featureColors = {
  aiAdvisor: {
    primary: '#2196f3',    // Blue
    secondary: '#64b5f6',
    background: '#e3f2fd'
  },
  methodsGenerator: {
    primary: '#9c27b0',    // Purple
    secondary: '#ba68c8',
    background: '#f3e5f5'
  },
  metaAnalysis: {
    primary: '#ff9800',    // Orange
    secondary: '#ffb74d',
    background: '#fff3e0'
  },
  studyDesign: {
    primary: '#4caf50',    // Green
    secondary: '#81c784',
    background: '#e8f5e9'
  },
  certification: {
    bronze: '#cd7f32',
    silver: '#c0c0c0',
    gold: '#ffd700',
    professional: '#e5e4e2'
  }
};
```

8.2 Component Patterns

```
- Floating AI chat button (bottom-right corner)
- Wizard step indicators (horizontal stepper)
- Side-by-side editor/preview layouts
- Card-based feature selection
- Interactive forest/funnel plots
- Progress rings for certification
- Export dropdown menus
```

---

9. Testing Strategy

9.1 Test Coverage Requirements

Feature	Unit Tests	Integration	E2E	Target Coverage
AI Advisor	80%	Yes	Yes	85%
Methods Generator	90%	Yes	Yes	90%
Meta-Analysis	95%	Yes	Yes	95%
Study Design	85%	Yes	Yes	85%
Certification	80%	Yes	Yes	85%

9.2 Validation Requirements

- Meta-analysis results validated against R's `metafor` package
- Effect size calculations validated against G*Power
- Methods templates reviewed by statisticians
- AI responses evaluated for accuracy

---

10. Deployment & DevOps

10.1 Infrastructure Requirements

```
Production Environment:
  Frontend:
    - Vercel or AWS CloudFront
    - CDN for static assets
    - Service worker for offline

  Backend:
    - AWS ECS or Kubernetes
    - Auto-scaling: 2-10 instances
    - Load balancer: AWS ALB

  Database:
    - PostgreSQL (AWS RDS)
    - Read replicas: 2
    - Redis cache cluster

  Storage:
    - S3 for file storage
    - CloudFront CDN

  AI:
    - Claude API (Anthropic)
    - Rate limiting: 100 req/min/user
```

---

11. Monetization Strategy

11.1 Pricing Tiers

```
FREE (Forever)
├── All 49 educational lessons
├── Basic analysis (1,000 rows)
├── 5 AI advisor queries/day
├── Community support
└── Bronze certification

PRO ($15/month or $150/year)
├── Everything in Free, plus:
├── Unlimited data size
├── Unlimited AI advisor
├── Methods section generator
├── R/Python code export
├── Meta-analysis (up to 50 studies)
├── Priority support
└── Silver certification access

TEAM ($50/month per seat)
├── Everything in Pro, plus:
├── Real-time collaboration
├── Team workspaces
├── Admin dashboard
├── SSO integration
├── API access
└── Gold certification access

ENTERPRISE (Custom pricing)
├── Everything in Team, plus:
├── Unlimited seats
├── On-premise deployment option
├── Custom integrations
├── SLA guarantee
├── Dedicated support
├── Compliance features (HIPAA, GDPR)
└── Professional certification program

ACADEMIC (Free for verified)
├── Pro features for students/faculty
├── LMS integration
├── Course management
└── Bulk certification
```

---

12. Timeline & Milestones

12.1 Phase 1: Foundation (Weeks 1-8)

```
Week 1-2: AI Statistical Advisor - Foundation
├── Claude API integration
├── Basic chat UI
├── Prompt engineering
└── Test Q&A functionality

Week 3-4: AI Statistical Advisor - Data Integration
├── Connect to data upload
├── Context extraction
├── Guardian integration
└── Test recommendations

Week 5-6: Methods Section Generator
├── Template system
├── APA 7 templates
├── Text generation
└── Export functionality

Week 7-8: Polish & Launch
├── UI/UX refinement
├── Performance optimization
├── Documentation
└── Beta testing
```

12.2 Phase 2: Core Features (Weeks 9-20)

```
Week 9-12: Meta-Analysis Module
├── Core calculations
├── Forest plots (D3)
├── Funnel plots
├── Heterogeneity analysis
└── Publication bias tests

Week 13-16: Study Design Wizard
├── Wizard framework
├── Variable builder
├── Power integration
├── Protocol generator
└── Pre-registration templates

Week 17-20: Certification Program
├── Progress tracking
├── Exam system
├── Certificate generation
├── Badge integration
└── LinkedIn badges
```

12.3 Phase 3: Advanced Features (Weeks 21-32)

```
Week 21-24: R/Python Code Export
Week 25-28: Multi-Language Support
Week 29-32: Mobile App (React Native)
```

---

13. Success Metrics

13.1 Key Performance Indicators

Metric	Current	6 Month Target	12 Month Target
Monthly Active Users	-	10,000	50,000
Lessons Completed	-	100,000	500,000
AI Queries/Month	-	50,000	250,000

Metric	Current	6 Month Target	12 Month Target
Methods Generated	-	5,000	25,000
Meta-Analyses Created	-	1,000	10,000
Certifications Earned	-	2,000	15,000
Pro Subscribers	-	500	3,000
NPS Score	-	50+	70+

---

14. Risk Assessment

14.1 Technical Risks

Risk	Probability	Impact	Mitigation
AI hallucinations	Medium	High	Validation layer, user feedback
Meta-analysis accuracy	Low	Critical	Validation against R packages
Scalability issues	Medium	Medium	Load testing, auto-scaling
Claude API downtime	Low	High	Fallback responses, caching

14.2 Business Risks

Risk	Probability	Impact	Mitigation
Low adoption	Medium	High	Marketing, SEO, partnerships
Competition	Medium	Medium	Unique features, quality
Monetization failure	Medium	High	Freemium model, academic focus

---

15. Session Continuation Notes

15.1 Current Implementation Status

```
 COMPLETED:
- Power Analysis Education Module (11 lessons)
- All existing modules (PCA, CI, DOE, SQC, Probability)
- Guardian validation system
- 46 statistical tests
- Complete button bug fixes
- Plot parameter visualization fix

 IN PROGRESS:
- Master Plan Document (THIS FILE)

 NEXT STEPS:
1. Create AI Statistical Advisor directory structure
2. Set up Claude API integration
3. Build basic chat UI
4. Implement prompt templates
5. Test with sample queries
```

15.2 Files to Create in Next Session

```
Priority 1 (AI Advisor):
/frontend/src/components/ai-advisor/
├── AIAdvisorHub.jsx
├── AIAdvisorChat.jsx
├── hooks/useAIAdvisor.js
└── utils/promptTemplates.js

/backend/ai_advisor/
├── views.py
├── services/ai_advisor_service.py
└── urls.py

Priority 2 (Methods Generator):
/frontend/src/components/methods-generator/
├── MethodsGeneratorHub.jsx
├── templates/apa7.js
└── utils/textGenerator.js
```

15.3 Environment Setup Needed

```
# Claude API key (add to .env)
CLAUDE_API_KEY=your_api_key_here

# Backend dependencies
pip install anthropic

# Frontend dependencies (if needed)
npm install @anthropic-ai/sdk  # Or use REST API
```

15.4 Key Decisions Made

1. **AI Model**: Claude (Anthropic) for statistical advice
2. **Meta-Analysis**: Pure Python implementation (validated against metafor)
3. **Certification**: 4-tier system (Bronze/Silver/Gold/Professional)
4. **Monetization**: Freemium with $15/month Pro tier
5. **Timeline**: 8 weeks to first major feature launch

Appendix A: Reference Materials

A.1 Statistical References

- Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences
- Borenstein, M. et al. (2009). Introduction to Meta-Analysis
- APA Publication Manual (7th Edition)

A.2 Technical References

- React 18 Documentation
- Django REST Framework
- D3.js Gallery (Forest Plots)
- Anthropic Claude API Documentation

Document maintained by: StickForStats Development Team Last review: December 12, 2025